

Apex Inventory

UI Requirement Specification

Replenish Orders

Route: /orders

Document Version: 1.0.0

Project: ReactExample (Apex Store Inventory Management)

1. Overview

The Replenish Orders page manages supplier replenishment orders. Users can place new orders for existing or new products, track pending deliveries, mark orders as received (which auto-updates inventory), cancel pending orders, and view completed order history.

2. Purpose

- Schedule future stock replenishment from suppliers.
- Track in-transit orders with expected arrival dates.
- Receive deliveries and automatically update inventory quantities.
- Maintain audit trail of completed orders.

3. Layout

- Top row: Place Order form (left) + Pending Deliveries list (right)
- Bottom: Completed Orders History table (full width)

4. Place Order Form

4.1 Order Item Selection

Segmented control (when items exist): 'Restock Existing Item' | 'Order New Product'. Defaults to select mode; switches to type mode if no items exist.

4.2 Item Input

- Select mode: dropdown with item name and current quantity.
- Type mode: text input for new product name.

4.3 Category

Dropdown from categories. Disabled when restocking existing item (auto-filled).

4.4 Physical State

- Select mode: read-only badge from item.
- Type mode: Solid / Liquid segmented control.

4.5 Quantity to Order

Number input, min 1, default '50'.

4.6 Total Cost

Number input with dollar icon, min 0, default '100'.

4.7 Expected Arrival Date

Date picker, required. Defaults to 5 days from today on page load.

4.8 Submit

Button: 'Place Replenish Order'. Creates order with status 'pending'.

5. Form Validation

- Select mode: valid item must be selected.
- Type mode: product name required.
- Category required.
- Quantity must be positive number.
- Price must be non-negative number.
- Expected arrival date required.

Success alert: 'Replenish order placed successfully!' Form resets quantity to 50, price to 100.

6. Pending Deliveries Panel

Lists all orders with status 'pending'. Each card shows:

- Item name and category badge
- Quantity, physical state, cost
- Expected arrival date
- Receive button - marks order delivered

- Cancel button - deletes pending order

Scrollable (max 420px). Empty state: 'No orders in transit'.

7. Receive Order Behavior

On 'Receive' click (POST /api/orders/:id/deliver):

- Order status changes to 'delivered'.
- If matching inventory item exists (by name): quantity increased by order quantity, lastStockIn updated.
- If no matching item: new inventory item created with 30-day default expiry, default analytics data.

Success alert: 'Order marked as delivered! Inventory stock has been automatically updated.'

8. Cancel Order Behavior

Confirm dialog: 'Are you sure you want to cancel this order?' DELETE /api/orders/:id. Order removed from system. No inventory change.

9. Completed Orders History

Table of delivered orders with columns:

Column	Data
Product Name	order.itemName
Category	order.category
Quantity Received	order.quantity
Physical State	solid/liquid badge
Cost Paid	order.price (\$)
Ordered On	order.orderDate
Arrival Date	order.expectedArrival
Status	Delivered badge

Empty state: 'No order history' when no delivered orders exist.

10. Auto-Population

When existing item selected in restock mode, category and physical state auto-populate. First category auto-selected when categories load and none selected.

11. Data Dependencies

- GET /api/orders, /api/categories, /api/items - load data
- POST /api/orders - place order
- POST /api/orders/:id/deliver - receive delivery

- DELETE /api/orders/:id - cancel order

12. Technical Notes

- Component: src/pages/PlaceOrder.tsx
- Route alias in nav: 'Replenish Orders' at /orders